

CS 3081 · Artificial Intelligence · Fall 2026

Assignment 1 — Search Strategies

Version B · BFS · DFS · Greedy Best-First · A*

Dr Naila Marir

Faris Almahdi

S24209780

Instructor: Dr Naila Marir
Section: 3081-2 (CRN 10481)

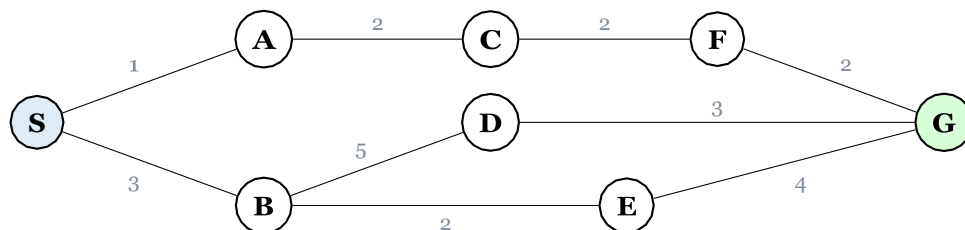
Total marks: 100

Instructions. Answer all four questions. Show your working — a correct final answer with no reasoning earns partial credit only. Where a question asks for the order in which nodes are expanded, write the nodes as a sequence, e.g. *S, A, C, G*.

- When two nodes have equal priority, break ties **alphabetically**.
- A node is *expanded* when it is removed from the frontier and its successors are generated. Counting the goal test, a node is expanded even if it turns out to be the goal.
- Assume the **graph-search** form of each algorithm: never expand the same state twice.
- This is an individual assignment. You may discuss ideas, but the work you submit must be your own.

1 Uninformed search: BFS and DFS

Consider the following state graph. The start state is *S* and the goal state is *G*. Edges are undirected, and the number on each edge is its step cost.



For this question ignore the edge costs. BFS and DFS are *uninformed*: they do not look at cost or at any estimate of distance to the goal.

(a) Breadth-first search

[8 marks]

List the order in which states are expanded by BFS, and give the path it returns.

- **Order of expansion:** S, A, B, C, D, E, F, G
- **Path returned:** S – B – D – G

(b) Depth-first search

[8 marks]

List the order in which states are expanded by DFS, and give the path it returns. Assume successors are pushed onto the stack in reverse alphabetical order, so that the alphabetically first successor is expanded next.

- **Order of expansion:** S, A, C, F, G
- **Path returned:** S – A – C – F – G

(c) Comparison

[6 marks]

BFS and DFS returned different paths. State which path is shorter in number of steps, and explain why BFS is guaranteed to find that path while DFS is not.

- **Shorter path in number of steps:** The BFS path (**S – B – D – G**, 3 steps vs. DFS's 4 steps).

- Answer:*
- **Explanation:** BFS searches outward level-by-level (by depth), guaranteeing the path with the fewest edges/steps. DFS explores along a single branch to maximum depth without regard to path length.

(d) Frontier data structures

[3 marks]

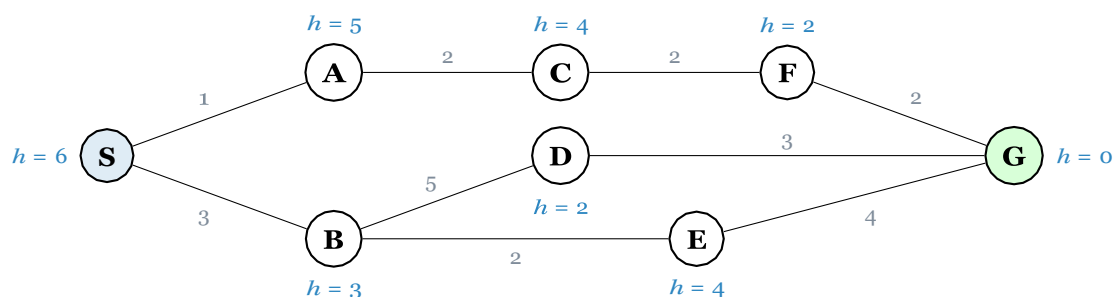
BFS and DFS differ by one design choice. Name the data structure each uses for its frontier, and explain in one sentence how that choice produces the difference in behaviour you observed.

- BFS uses a **FIFO queue**; DFS uses a **LIFO stack**.

- Answer:*
- **Explanation:** A FIFO queue prioritizes shallowest nodes first, whereas a LIFO stack explores newly generated deep successors first.

2 Informed search: Greedy Best-First and A*

The same graph is repeated below. Each state is now annotated with a heuristic value $h(n)$ — an estimate of the cost from n to the goal G .

**(a) Greedy best-first search**

[8 marks]

Greedy best-first expands the frontier node with the smallest $h(n)$. List the order of expansion and the path returned.

- **Order of expansion:** S, B, D, G
- **Path returned:** S – B – D – G

(b) A* search

[12 marks]

A* expands the frontier node with the smallest $f(n) = g(n) + h(n)$. Complete the table for every node *at the moment it is expanded*, then give the order of expansion and the path returned.

Order	Node	$g(n)$	$h(n)$	$f(n)=g(n)+h(n)$
1	S	0	6	6
2	A	1	5	6
3	C	3	4	7
4	F	5	2	7
5	G	7	0	7

- Order of expansion: S, A, C, F, G
- Path returned: S – A – C – F – G
- Total cost: 7

(c) Why they differ

[6 marks]

Greedy best-first and A* returned different paths, or explored a different number of nodes, or both. Explain the difference by referring to what each algorithm ignores.

Greedy Best-First ignores the accumulated path cost $g(n)$, selecting locally appealing nodes based solely on $h(n)$ which results in a suboptimal path (cost 11). In contrast, A* balances both actual cost and estimated remaining cost via $f(n) = g(n) + h(n)$, guaranteeing the least-cost path.

3 Admissibility and optimality**(a) Definition**

[5 marks]

Define an **admissible** heuristic. Your definition must be precise enough that someone could check a given heuristic against it.

A heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal:

$0 \leq h(n) \leq h^*(n)$ for all nodes n

where $h^*(n)$ is the true minimum cost from node n to the goal G , and $h(G) = 0$.

(b) Testing the heuristic

[10 marks]

For the graph in Question 2, compute the true cost $h^*(n)$ of the cheapest path from each node to G , and decide whether the given h is admissible.

Node	S	A	B	C	D	E	F	G
$h(n)$ given	6	5	3	4	2	4	2	0
$h^*(n)$ true	<u>7</u>	<u>6</u>	<u>6</u>	<u>4</u>	<u>3</u>	<u>4</u>	<u>2</u>	<u>0</u>
$h \leq h^*$?	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>

Is h admissible? Yes _____

If not, name every node that violates the condition: none _____

(c) Consequence

[6 marks]

Suppose a heuristic *overestimates* the true cost at exactly one node. Explain what this can do to the solution A^* returns, and why uniform-cost search (A^* with no heuristic) could never suffer from this problem.

Overestimating $h(n)$ at a node makes an optimal path through that node appear artificially expensive, which can cause A^* to abandon it and return a suboptimal path. Uniform-cost search avoids this because it uses $h(n) = 0$ everywhere (never overestimating non-negative step costs) and expands nodes strictly in increasing order of $g(n)$.

(d) A perfect heuristic

[3 marks]

Suppose $h(n) = h^*(n)$ for every node. How does A^* behave in this case? Justify your answer in one sentence.

A^* expands only the nodes along the optimal path(s) directly to the goal with no wasted node expansions, because all non-optimal paths have strictly larger f -values.

Answer:

4 Comparing the four strategies

(a) Properties table

[8 marks]

Complete the table. Name the data structure each strategy uses for its frontier. For *complete* and *optimal*, write Yes or No and state any condition required for that answer to hold.

Strategy	Frontier	Complete?	Optimal?
Breadth-first	FIFO queue	Yes (if branching factor b is finite)	Yes (if all step costs are equal)
Depth-first	LIFO stack	Yes (for finite state spaces / graph search)	No
Greedy best-first	Priority queue (by h)	Yes (for finite state spaces in graph search)	No
A^*	Priority queue (by $f = g + h$)	Yes (if b is finite and step costs $\geq \epsilon > 0$)	Yes (if h is admissible / consistent)

(b) Choosing a strategy

[13 marks]

For each scenario, name the most appropriate of the four strategies and justify your choice. Refer to the properties in part (a) — a bare name earns no marks.

- A delivery drone must find the cheapest flight path between two warehouses, and the straight-line distance to the destination is known for every waypoint.
- A web crawler must find *any* page containing a keyword. The link graph is very deep, memory is tight, and the shortest link path is not important.
- A game AI must reach a target square quickly on a large map. A good distance estimate exists, but the move does not need to be the cheapest — only fast to compute.

- **i. Delivery drone: A^* search.** Straight-line distance is an admissible heuristic, and A^* guarantees finding the cheapest (optimal) path while minimizing expansions.
- **ii. Web crawler: Depth-first search (DFS).** Memory is tightly constrained, path length is irrelevant, and DFS requires only $O(bm)$ linear memory compared to BFS's exponential memory.
Answer:
- **iii. Game AI: Greedy best-first search.** Optimality is not required, while speed is critical; Greedy search uses the heuristic to rapidly advance toward the target square with low computational overhead.

(c) Short answer

[4 marks]

BFS is complete and optimal for uniform step costs, but is often impractical on large problems. State the main reason, and name one property of the search problem that makes it worse.

- **Main reason BFS is impractical: Exponential memory consumption** ($O(b^d)$ space complexity).

Answer:

- **Property making it worse:** A **high branching factor** (b) (or large search depth d).

Submission. Submit a single PDF to Blackboard by the due date. Handwritten work is acceptable if scanned clearly. Include your name and student ID on the first page.

Q1: 25 Q2: 26 Q3: 24 Q4: 25 **Total: 100 marks**